

WasmAndroid: A Cross-Platform Runtime for Native Programming Languages on Android

ELLIOTT WEN, GERALD WEBER, and SURANGA NANAYAKKARA,

The University of Auckland, New Zealand

Open source hardware such as RISC-V has been gaining substantial momentum. Recently, they have begun to embrace Google's Android operating system to leverage its software ecosystem. Despite the encouraging progress, a challenging issue arises: a majority of Android applications are written in native languages and need to be recompiled to target new hardware platforms. Unfortunately, this recompilation process is not scalable because of the explosion of new hardware platforms. To address this issue, we present WasmAndroid, a high-performance cross-platform runtime for native Android applications. With WasmAndroid, developers can compile their source code to WebAssembly, an efficient and portable bytecode format that can be executed everywhere without additional reconfiguration. Developers can also transpile existing application binaries to WebAssembly when source code is not available. WebAssembly's language model is very different from other common languages. This mismatch leads to many unique implementation challenges. In this article, we provide workable solutions and conduct a thorough system evaluation. We show that WasmAndroid provides acceptable performance to execute native applications in a cross-platform manner.

CCS Concepts: • **Computer systems organization** → **Embedded software**;

Additional Key Words and Phrases: Android, WebAssembly, cross-platform, open source hardware

ACM Reference format:

Elliott Wen, Gerald Weber, and Suranga Nanayakkara. 2022. WasmAndroid: A Cross-Platform Runtime for Native Programming Languages on Android. *ACM Trans. Embedd. Comput. Syst.* 22, 1, Article 4 (October 2022), 19 pages.

<https://doi.org/10.1145/3530286>

1 INTRODUCTION

Recently, the open source hardware movement has established a solid foothold. Exemplary **Instruction Set Architectures (ISA)** include RISC-V (**reduced instruction set computing instruction set (RISC)**) [38], OpenRISC [37], and Microwatt [8]. They possess various advantages (e.g., license-free, highly modular, and easily extensible) and are ushering in a new era of silicon design and processor innovation. Lately, these ISAs have begun to support Google's Android operating system [1]. Android is open source and widely used in embedded platforms and industry sectors. It also possesses a mature software ecosystem. The combination of open source hardware

Authors' addresses: E. Wen, G. Weber, and S. Nanayakkara, The University of Auckland, 70 Symonds Street, Grafton, Auckland, New Zealand; emails: jq.elliott.wen@gmail.com, g.weber@aucklanduni.ac.nz, suranga@ahlab.org.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2022 Association for Computing Machinery.

1539-9087/2022/10-ART4 \$15.00

<https://doi.org/10.1145/3530286>

and Android provides a potential escape route for many mobile device vendors wary of proprietary licensing pitfalls.

However, to fully embrace open source platforms, Android developers have yet to overcome a key challenge: a considerable number of Android applications (39.7%) are written in native languages (C/C++) and need to be recompiled to target new ISAs [39]. This process may not be scalable considering open ISAs are fast emerging and evolving. What makes the problem worse is that the open ISAs allow chip designers to customize instruction extensions. For example, they can decide whether to include floating-point units, **Single Instruction Multiple Data (SIMD)**, or other coprocessors to trade off hardware resources and computing performance. For each configuration, the instruction sets can be different, and one corresponding native binary needs to be prepared.

To address these issues, we present WasmAndroid, a high-performance cross-platform runtime for native Android applications. With WasmAndroid, developers do not need to generate native binaries for every CPU architecture. Instead, they can compile source codes into WebAssembly, a bytecode format recently invented by mainstream browser vendors [18]. WebAssembly is entirely agnostic to hardware architectures and operating systems. It thus allows Android applications to be compiled once and executed everywhere without any additional reconfiguration. Unlike other high-level languages bytecode, the WebAssembly instruction set is closely related to modern processors' instruction sets. Therefore, a WebAssembly binary can be efficiently translated into native CPU instructions and deliver an execution speed as fast as native. It is worthwhile to mention that WasmAndroid can also transpile many existing application binaries (presumably ARM) to WebAssembly. This feature could be helpful when we do not have access to an application's source code.

However, incorporating WebAssembly with Android entails many technical challenges. First, WebAssembly programs expect a linear memory model, where memory content is laid out contiguously starting from address 0. This model is very different from Android's memory model, where an application is usually allocated in a random high memory address by default. Second, WebAssembly discards the support to raw function pointers in C/C++ to pursue higher memory security. This feature can be problematic, because the Android Java runtime frequently communicates with native code using raw function pointers. Third, WebAssembly is fully sandboxed and does not have built-in instructions for system calls. Developers have to expose system call functions to WebAssembly applications explicitly. By exposing fewer functions, we can reduce the attack surface of native code, but some applications may also stop working as a result of missing system calls. It is tricky to strike a balance between functional correctness and security for all applications.

In this article, we present practical solutions to cope with the above challenges. To address the impedance mismatch in memory models, our WebAssembly runtime can generate necessary instructions that map WebAssembly memory addresses to the Android memory space. This mapping overhead can even be avoided with a slight modification in the Android kernel. To interoperate with Android Java runtime, our system can identify functions that may receive raw function pointers and conduct necessary code transformations. In addition, our system allows developers to use the simple script language Lua to specify necessary system calls for each application. It also allows dynamic altering of system call behavior, which can be used to implement advanced security features such as the live patching of kernel-level vulnerabilities.

We consolidated the above techniques and implemented a prototype of WasmAndroid on a RISC-V simulator. We then conducted a thorough performance evaluation using SPEC CPU 2006 benchmark suites. The experiment results show that WasmAndroid provides acceptable performance to execute native applications in a cross-platform manner. The main contributions of this article can be summarized as follows:

- (1) We propose a framework that allows native Android applications to be compiled into WebAssembly once and run in every CPU architecture.
- (2) We implement an Android-optimized WebAssembly compiler, which can efficiently interoperate with Android Java runtime and provide software-based fault isolation.
- (3) We conduct a quantitative performance evaluation on our WasmAndroid prototype on a simulator.

We initially presented WasmAndroid at the 22nd ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems [42]. In this article, we have substantially expanded the technical methodology and conducted an extensive system evaluation. The rest of the article is organized as follows. Section 2 describes the background. Section 3 describes in detail WasmAndroid architecture. We present evaluation results of our approach in Section 5, extensions and limitations of WasmAndroid in Section 6, and conclude in Section 7.

2 BACKGROUND AND RELATED WORK

To facilitate readers' understanding of our article, we provide some background knowledge on Android, open source hardware, and WebAssembly in this section. They also serve as motivation to build a cross-platform runtime for native Android applications using WebAssembly.

2.1 High-level Programming Languages in Android

Android applications can be written in Java or Kotlin [31]. These high-level languages are portable, because they are compiled to intermediate bytecodes, which are universally interpretable by different Java virtual machines (e.g., Android Art or Dalvik [2]). However, the use of Java virtual machines leads to the following two issues.

Degraded Performance. Java or Kotlin code generally run slower than native programming languages such as C or C++. One reason is that they need to go through an extra interpretation stage. To suppress the overhead, Android adapts several sophisticated compiler technologies such as threaded interpreters and just-in-time compilers. Nevertheless, the performance gap is still not negligible [27]. Another bigger culprit is that Java virtual machines adopt automatic garbage collectors, which will keep track of each object in memory behind the scene. When an object is no longer being used, it will be released automatically. This frees programmers from error-prone memory management but at the cost of a higher CPU workload and memory footprint. In worst-case scenarios, the garbage collectors may need to pause the application execution for a considerably long time (in the order of milliseconds). This latency may not be acceptable to certain types of applications (e.g., three-dimensional games).

Prone to Reverse Engineering. Reverse engineering is the process of gaining information about a program and its implementation details. Sometimes, it is for corporate espionage or intellectual property theft. Unfortunately, Android Java bytecode is an easy target of the reverse engineering attack due to its informant and well-structured syntax. A considerable number of Android reverse engineering tools (e.g., APKTool [43] and Procyon [16]) are readily available. They can be used to dump the executable code and even reconstruct the source code.

2.2 Native Programming Languages in Android

To implement computationally intensive components or prevent reverse engineering, Android allows the use of native programming languages such as C or C++. According to a recent study [39], approximately 40% of Google Play applications have employed native code.

To facilitate the development process, Android provides a **native development toolkit (NDK)** [34], which are essentially platform libraries and C/C++ compilers. An NDK allows developers

to incorporate native code in two paradigms. In the first mode, developers can implement certain functions in native languages and bundle the compiled binary as a shared object into the application. Java code can dynamically load and invoke the compiled functions using **Java Native Interface (JNI)**. JNI defines how Java code sends data to native functions and receives return values. It also specifies how native code accesses Java objects and invokes Java methods. In the other paradigm, developers can implement an Android Activity component entirely in native code. This type of component is referred to as *Native Activity*. It is worth noting that despite the absence of Java source code, the Java compiler is still required to generate hidden executable stubs to facilitate application startup. Native activity applications are rare and mainly used in gaming engines. In this article, we will mainly discuss the former paradigm.

A significant downside of NDK is that the native binaries is not portable across Android devices with different processors. This was only a minor concern, since there were only two dominant mobile CPUs: X86-64 and Arm64-v8a. Developers could still ensure portability by bundling binaries for both the architectures in application installation files.

Recently, the open source hardware movement has been gaining substantial momentum [8, 37, 38]. A latest advancement is that Alibaba has successfully ported Android 10 to the RISC-V instruction set architecture [6]. The fast-emerging open source processors now extensively aggravate the portability issue of NDK. The old “bundle-all” strategy would render the installation files very bulky and fail to scale in the near future. What makes the matter worse is that open hardware allows chip designers to customize instruction sets. Take RISC-V as an example. RISC-V features a modular design; it consists of base parts and extensions as shown in Table 1. The base specifies instructions encoding, register sets, memory addressing schemes, and logic/control flow commands. The base alone can implement a simplified general-purpose computer with full software support. However, RISC-V extensions are optional and designed to enhance the performance of the bases. It can be seen that there are multiple combinations between the bases and the extensions. For each configuration, their instruction sets may not be compatible and one corresponding native library is also needed. For instance, apps that utilize the vector processing instructions would not run on processors without such an extension.

To address this issue, we propose a cross-platform runtime for native programming languages on Android. The main idea is that instead of generating native binaries for every CPU architecture, we can compile native source code into a particular kind of intermediate representation. In analogy to Java bytecode, this representation should be interpretable by different machines. Moreover, it should provide source code protection and execution performance comparable to native binaries. In this article, we select “WebAssembly” as our intermediate representation.

2.3 WebAssembly

WebAssembly is a young binary instruction format first released in 2017 by mainstream browser vendors [18]. Its primary design goal was to provide an approach to run portable executables with near-native performance in web browser environments. Nowadays, there has been an emergence of web applications built on WebAssembly, for example, cryptocurrency [28], computer vision [36], emulators [40], and games [45]. More recently, researchers have started taking WebAssembly beyond the web. Several previous works [19, 23, 41] have explored WebAssembly in the context of security and portability for edge computing and IoT devices.

WebAssembly’s popularity is attributed to the following three unique advantages:

- (1) **Compile Different Languages to WebAssembly and Run Everywhere.** WebAssembly bytecode is entirely agnostic to hardware architectures and operating systems. This means that a program can be compiled into WebAssembly once and run across different hardware

Table 1. Exemplary RISC-V Bases and Extensions

Bases	
RV32I	32-bit integer instruction set
RV32E	Simplified 32-bit integer instruction set optimized for embedded devices
RV64I	64-bit integer instruction set
RV128I	128-bit integer instruction set
Extensions	
M	Multiplication/division instructions
C	Compacted instructions for reducing power consumption, code size, and memory use
F	Single precision floating point instructions
D	Dingle precision floating point instructions
V	Vector processing instructions (SIMD)

architectures and operating systems without any additional reconfiguration. In addition, WebAssembly is an open standard and is designed to be an effective compilation target for almost every source language. Nowadays, commonly-used language compilers (e.g., C, C++, Rust, Go, and Dart) can already generate WebAssembly as the final output.

- (2) **Sandboxed Execution.** WebAssembly provides a sandboxing solution for running untrusted code. First, WebAssembly applies a linear memory model, where an application's memory is laid out linearly and fixed at compile time. This can effectively prevent common security vulnerabilities arising from direct memory access. Second, WebAssembly features structured control flow, which disallows the use of several erroneous control flow instructions such as *goto* statements and raw function pointers. This prevents malicious programs from performing unsafe branching to hijack application control flow. Third, WebAssembly does not have built-in privileged instructions such as system calls and hardware input/output. These sensitive operations can only be carried out with the help of external interfaces explicitly exposed by the runtime. We can easily scrutinize these external interfaces to improve system security. For instance, we can blocklist file system interfaces to prevent a WebAssembly program from accessing local disk files.
- (3) **Promise of Native Performance.** The WebAssembly specification promises an execution speed as fast as native. It has been primarily achieved through WebAssembly's well-engineered instruction set, which shares high similarities to modern physical processor instruction sets. This ensures that a WebAssembly program can be quickly and efficiently transformed to native machine code. Being a young technology, WebAssembly still has some room for improvement. As reported by previous benchmarks [24], applications compiled to WebAssembly may run slower than their native counterparts due to poor compiler designs and absent instruction extensions. Nevertheless, many efforts have been made lately to narrow the performance gaps. For instance, Hall et al. [19] enhanced Google's V8 WebAssembly compiler to enable high-performance serverless computing in resource-constrained devices. The WebAssembly SIMD extensions are also being proposed to take advantage of the data level parallelism using the underlying hardware [11].

3 SYSTEM ARCHITECTURE

Figure 1 demonstrates the main workflow of WasmAndroid. Given a native Android application's source code, developers can compile it into WebAssembly bytecode using a standard WebAssembly

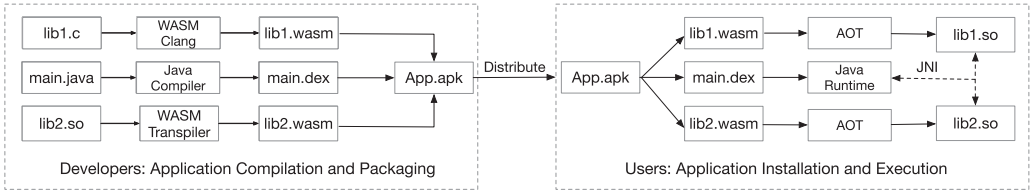


Fig. 1. WasmAndroid enables native code to be compiled once and execute everywhere.

toolchain called *Clang-Wasm64* [3]. Alternatively, given an existing binary file, developers may transpile it to WebAssembly directly (see Section 6.1). The generated WebAssembly bytecode will be embedded in an **android application package (APK)** and distributed to mobile devices with different CPU architectures. This WebAssembly generation process needs only to be carried out once.

To install such an APK, a patched Android device will first extract the WebAssembly bytecode. The bytecode is hardware independent and not directly executable by physical CPUs. It has to be fed into a WebAssembly runtime to generate native CPU instructions before execution. A WebAssembly runtime can be implemented using an interpreter, which translates and executes one WebAssembly instruction at a time. A runtime can also be implemented in an **ahead-of-time (AOT)** compilation manner, where a WebAssembly program is transformed into native machine code at once before it runs. WasmAndroid provides both of the implementations. By default, WasmAndroid adopts the AOT compiler, since it can perform complex code optimizations to generate highly efficient binary objects. The resulting objects can then be dynamically invoked by the Android Java runtime using JNI.

4 AOT COMPILATION FOR WEBASSEMBLY

A core component of WasmAndroid is the AOT WebAssembly compiler. In this section, we will illustrate the implementation challenges and solutions.

4.1 Instruction Translation

We construct an AOT compiler for WebAssembly with the help of LLVM compiler infrastructure [29]. In detail, we first implement a lexer and a parser for WebAssembly bytecode using Flex and Bison [30]. These tools enable us to iterate each WebAssembly instruction and convert it into **LLVM intermediate representation (LLVM IR)**. The conversion is generally not difficult, since WebAssembly and LLVM IR are low-level instructions, and many of them are semantically similar. One difficulty we encounter is that LLVM IRs are register based (i.e., operands and results are stored in registers), but WebAssembly instructions are stack based (i.e., pops operands from and pushes results onto a stack). To conduct the conversion, we need to maintain a stack structure to store the mapping between the WebAssembly operands and LLVM registers. It should be noted that WebAssembly adopts structured control flow [18], which allows us to statically determine WebAssembly operands' memory locations on compile time. As a result, the stack structure only exists during compilation and would not incur runtime overhead. The direct conversion result is generally suboptimal and can be extensively improved using LLVM optimization passes such as register renaming, constant folding/propagation and loop unrolling. The optimized IRs are then sent to LLVM backends to generate native objects for the target CPU architecture. From the Android Java runtime's perspective, the resulting objects behave the same as the native libraries generated using NDK.

Listing 1: Transforming a WebAssembly address to a valid OS address.

```

1: %1 = load i8*, i8** linear_mem
2: %transformed_addr = getelementptr inbounds i8, i8* %1, i64 100
3: call @runtime.bounds_check(i8* %transformed_addr)
4: %2 = bitcast i8* %transformed_addr to i32*
5: %result = load i32, i32* %2

```

4.2 Memory Mapping

Each WebAssembly application expects one linear memory, which is byte addressable and spanning from address 0 to mutable memory size. The memory content can be accessed by memory instructions “load” and “store” with an index parameter. For example, a WebAssembly instruction “i32.load 100” will load a 32-bit integer located in memory locations 100–103. This is very different from the memory model of real-world OSs, where applications are randomly allocated in a high virtual memory address space (e.g., 0xffffffff80000000). To bridge the difference, our compiler generates necessary instructions to map each WebAssembly memory address to a valid memory address in the OS. In detail, our AOT compiler first generates a program initialization routine (i.e., ctor function) to allocate a contiguous memory chunk from the underlying OS. The region’s address is then recorded in a global variable *linear_mem*. Afterward, the compiler can offset each WebAssembly memory operation’s parameter with the value of *linear_mem*. For instance, the WebAssembly instruction “i32.load 100” will be translated into the simplified LLVM IR as shown in Listing 1. We also inject bounds-checking instructions to ensure that all memory operations are well-defined. If out-of-bound access is detected, then we can terminate the application immediately to ensure system safety.

However, such a transformation inevitably incurs a significant performance penalty, because it inserts more than 10 machine instructions for each memory operation. This article provides an alternative approach to avoid the expensive memory address transformation. The key idea is to use the system call “mmap” to allocate a memory region in a low virtual memory address space. The region starts at address 0 and has the same size as the WebAssembly linear memory. Hence, address transformation is not more required as all WebAssembly memory addresses are now valid OS addresses. In our implementation, we can set the linear memory size to a constant of 16 GB. This assumes a native application does not consume more than 16 GB of physical memory. This assumption generally holds, since even top-tier mobile devices are not equipped with that amount of memory. It should be noted that the 16 GB is just the size of virtual memory. It does not mean the application will actually consume 16 GB of physical memory at runtime. In fact, the system call “mmap” manages physical memory resources using the “Demand Paging” strategy. When the 16 GB virtual memory is allocated, the obtained memory pages initially refer to a special region filled with zeros. A physical memory page is not allocated until a memory write operation to that page is made. This allows processes to reserve more virtual memory than physical memory and use memory resources sparsely.

This “mmap” technique also facilitates compilers to optimize out certain bounds checking statements. Specifically, we exploit an optimization pass called *Value Range Analysis*, which tracks the range of values that a numeric variable can take. We compare the analysis results for each WebAssembly address with the constant 32 GB at compile time. If the upper bound of the result is lower than that constant, then we can safely eliminate the bounds checking statements.

A technical challenge is that the system call “mmap” generally disallows memory allocation starting from address 0 to prevent NULL pointer dereference exploits in kernel space. A naive workaround is to adjust the kernel parameter “mmap_min_addr,” which specifies the minimum

Table 2. WebAssembly Function Table

Index	Memory Address	Function Signature
0	foo (0xffffffff12341000)	(param i32) (result i32)
1	bar (0xffffffff45671000)	(param i32) (result i32)
2	...	...

Listing 2: Example C Source Code Interacting With Java Runtime.

```

1 // JNIEnv is allocated and initialized by Java Runtime.
2 typedef struct _JNINativeInterface {
3     ...
4     jint (*GetVersion)(JNIEnv *env);
5     ...
6     jobject (*ThrowNew)(JNIEnv *env, ...);
7     ...
8     jobject (*NewObject)(JNIEnv *env, ...);
9     ...
10 } JNIEnv;
11
12 // Native method to be invoked by Java Runtime
13 jint Java_com_foo_jni_bar_getVer(JNIEnv *env, jobject thisObject) {
14     // Get the runtime version.
15     return env->GetVersion(env);
16 }

```

virtual address that a process can allocate (4096 by default). This, however, may increase the security implications. In this article, we safely bypass the “mmap” restriction by applying minor modifications to the Clang toolchain. The patched toolchain now allows us to reserve a safeguard region of memory starting from address 0 using a compiler flag called “MemoryBase.” When we set the flag value to 4096, we can generate WebAssembly programs whose linear memory is filled with garbage data in locations 0–4095. Since the garbage data will never be referenced by any instructions, we do not need to load them to the physical memory. In other words, we are only required to “mmap” a memory region starting from address 4096.

4.3 Function Pointer Transformation

In C/C++, a function pointer is used to represent a function’s memory address. Though powerful, function pointers are culprits of many common security vulnerabilities (e.g., stack smashing and control-flow hijacking). WebAssembly discarded the support to raw function pointers. Instead, WebAssembly introduces a primitive called function table to manage functions’ memory addresses as illustrated in Table 2. In the table, every function is referenced by a unique index, which simulates a raw function pointer. To invoke such a function, for example “bar,” WebAssembly needs to issue the instruction “call_indirect” and supply its index 1 as the operand. Under the hood, the runtime will first bound check the index against the table. After that, it can retrieve and jump to the corresponding memory address. Note that the function table also stores signature information for each function. This allows us to verify whether the expected type of the indirect function call matches the actual type of the function to be called. This signature check is crucial for security, because it is undefined behavior in C++ to cast a function pointer to a random type and call it that way. This behavior can potentially smash stack data in some platforms.

Listing 3: X86-64 Assembly generated using NDK.

```

1 Java_com_foo_jni_bar_getVer :
2     ...
3     jmpq *32(%rdi)
4     ...
5     retq

```

Listing 4: WebAssembly generated using clang-wasm64.

```

1 .functype Java_com_foo_jni_bar_getVer (i64) -> (i32)
2     ...
3     local.get 0 // JNIEnv Address
4     i64.load 32
5     i32.wrap_i64
6     call_indirect
7     ...

```

However, this security feature leads to a unique challenge when we inter-operate native code with the Android’s Java runtime. Specifically, each native method directly invoked by Java (i.e., public JNI methods) will receive a “JNIEnv” (or “JavaVM”) as the first argument. As shown in Listing 2, JNIEnv is essentially a C structure managed by the Java runtime. It contains function pointers to interact with the Java runtime and Java objects (e.g., Java/native array conversion, objects initialization, and throwing exceptions). In Listing 2, we showcase the “GetVersion” function to obtain the version number of the Java virtual machine. When the source code is compiled using NDK, we can obtain semantically correct X86 machine code as shown in Listing 3. In detail, the register RDI holds the memory address of a JNIEnv structure. By offsetting the address by 32 bytes, we obtain the fifth element inside the structure (i.e., “GetVersion” memory address). The “jmpq” instruction will then perform an unconditional branch to that memory address to invoke “GetVersion.” However, in the WebAssembly world, the same code will be misinterpreted as shown in Listing 4. In Line 3 and 4, we again offset the JNIEnv address by 32 bytes to obtain “GetVersion” memory address. In Line 5 and 6, WebAssembly mistreats the address as a function table index. Since WebAssembly specification defines a table index to be 32-bit, the address is first truncated to a 32-bit integer and then fed to the “call-indirect” instruction. This illogical index will be caught by the function table bound check and trigger an out-of-bound runtime error.

To address this issue, we need to transform raw function pointers in the JNIEnv structure into function table indexes before passing them to the WebAssembly world. To achieve that, our AOT compiler generates a wrapper function for each native method as demonstrated in Listing 5. Specifically, the wrapper first iterates every raw function pointer in the incoming “JNIEnv” structure. If a raw function pointer does not exist in the function table, then it will be appended to the table. After that, the wrapper duplicates a “JNIEnv” structure and replaces every raw function pointer inside with the corresponding function index. The transformed “JNIEnv” will then be passed to the target method.

It is worth noting that developers can use two approaches to define public JNI methods. The first one is static registration, where developers follow a particular naming convention to declare their public methods. Specifically, a public method name is concatenated from the following components:

Listing 5: Transforming C pointers to valid WebAssembly function indexes.

```

1: function Java_com_foo_jni_bar_getVer_wrapper(JNIEnv *env):
2:     dupenv ← duplicate(env)
3:     foreach pointer in dupenv do
4:         if pointer in table then
5:             dupenv[pointer] = table.getIndex(pointer)
6:         else
7:             dupenv[pointer] = table.createIndex(pointer)
8:     call Java_com_foo_jni_bar_getVer(dupenv)

```

- (1) the prefix “Java_”
- (2) a mangled fully-qualified class name
- (3) an underscore separator (_)
- (4) a mangled method name
- (5) two underscores (__) followed by the mangled argument signature when the method is overloaded.

The alternative method is dynamic registration, where developers use a JNI API called “RegisterNative” to provide memory addresses of native methods at runtime. Compared with static registration, dynamic registration possesses unique advantages. It allows public methods to have arbitrary naming schemes, potentially improving the readability of source code. It also enables developers to modify native method implementation at runtime.

To locate these two types of public methods, we implement an LLVM *ModulePass* for the clang-wasm compiler toolchain. In detail, to locate static JNI methods, our pass will iterate all functions in a program and examine whether their name matches the static registration naming patterns. To locate dynamic JNI methods, our pass first needs to locate all call sites of the function “RegisterNative.” Afterwards, the pass applies data flow analysis [26]; it recursively climbs up the use-def lists from the parameters to find the definitions of all dynamic methods. If a dynamic method is found, then our pass also needs to generate a wrapper for the function “RegisterNative.” The reason is that in the WebAssembly world, all function pointers are represented as function indexes. Hence, the “RegisterNative” will receive function indexes as parameters. This function indexes, however, are not directly interpretable by Java runtime. Therefore, a wrapper is required to map the indexes back to memory addresses (i.e., a reverse process of Listing 5).

4.4 System Call Protection

WebAssembly bytecode has no built-in privileged instructions for system calls or hardware I/O. To carry out these security-sensitive operations, WebAssembly relies on a special “Imported Function Table” as shown in Figure 2. This table is initialized by the runtime when the application is starting up. The runtime can inject external function references (e.g., system calls) into it and they will be directly invocable by WebAssembly programs similar to typical functions. This mechanism effectively serves as a bridge between the WebAssembly world and the underlying operating system. Note that C/C++ developers generally would not directly invoke those low-level system call interfaces. Instead, they use a library called *libc* [13], which provides applications with **Portable Operating System Interfaces (POSIX)** for common tasks such as I/O and Filesystem. Behind the scene, the library will map the POSIX requests into one or more external functions in the imported table.

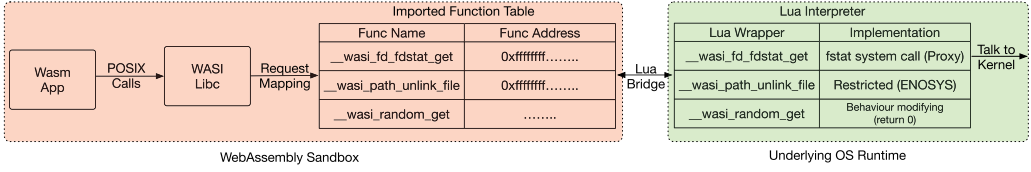


Fig. 2. System call protection using the Lua Interpreter. We showcase three wrapper implementation paradigms; the “unlink” system call is now disallowed to prevent WebAssembly programs from deleting any files. The “fstat” system call will function properly. The behavior of the “random” system call is changed such that it always returns zero.

This architecture allows us to implement a software fault isolation mechanism to reduce the attack surface of native code. The main idea is to expose as minimum as possible system calls to WebAssembly programs while keeping them functional. For example, if the native code only conducts mathematical computation, then the runtime needs not expose any external methods. Similarly, if the native code is not expected to access the Internet, then the runtime does not have to expose socket system calls. Since every native application may expect different system calls, it would be beneficial to allow application developers to specify their system call requirements. To achieve that, WasmAndroid harnesses the simplicity of Lua [22], a dynamically typed and memory-safe scripting language. Specifically, we allow developers to embed an optional Lua script as auxiliary information in the WebAssembly bytecode. In the script, developers need to implement a set of system call wrappers. These wrappers will be injected into the imported function table upon application startup. Wrappers could have three different implementation paradigms as showcased in Figure 2. The most straightforward paradigm is to return “ENOSYS,” indicating the system call is restricted. Alternatively, the wrapper can invoke the original system call and forward the parameters from WebAssembly as is. This wrapper then serves as a proxy to the system call. The most sophisticated paradigm is to modify the system call behavior programmatically. A typical usage is to examine the system call parameters and filter them if an attempt to exploit kernel vulnerabilities is detected. This feature potentially enables the live patching of kernel-level vulnerabilities.

5 SYSTEM EVALUATION

In this section, we provide a system evaluation on WasmAndroid. We are interested in one key question: How efficient are WebAssembly-based Android applications in CPU utilization and memory footprint? We use the following experiment setups to answer this question.

5.1 Test Bed Configurations

Hardware. We conduct the experiments on an emulated RISC-V machine, which consists of a quad-core RV64GV¹ CPU and 4 GB of RAM. The emulation is carried out using QEMU [15] running on a computer equipped with a 16-core 3.5 GHz AMD 3950x processor and 64 GB of 3,600-MHz DDR4 RAM.

Software. Our prototype is based on the Android Open Source Project Android 10 release. We apply a patch from T-Head [9] to bring support for the RISC-V architecture. This patch, however, is still at an experimental stage. We need to modify several source files to complete the building process. A pending technical issue is that the patched just-in-time Java compiler occasionally generates illegal RISC-V machine code and crashes application processes. To bypass this issue temporarily, we disable the just-in-time compiler and switch to the inefficient Java bytecode

¹Shorthand for RV64IMAFDZicsrZifenceiC.

interpreter. This workaround does not significantly impact the benchmarking accuracy, because our benchmarking applications only have a limited number of Java functions (mainly for loading the native code). The majority of CPU cycles are still spent on native code execution.

We also enhance the Android application installation subsystem (i.e., `PackageManagerService`) to handle APK files with embedded WebAssembly code. Specifically, we patch a file called “`NativeLibraryHelper.java`” to add a supported ABI “`wasm`” for every architecture. Therefore, when the installer extracts native binaries from an APK, it will first scan the directory “`lib/<native-abi>`” as usual. If the directory does not exist, then the installer will visit the “`lib/wasm`,” where the WebAssembly binaries are stored. Afterward, the installer can invoke our AOT compiler to convert the WebAssembly code into native binaries and copy them to application paths.

5.2 CPU Performance Measurement

As suggested by previous work on WebAssembly performance evaluation [24], we selected 13 benchmark executables from an established CPU benchmark suite *SPEC CPU 2006* [21]. They all have a modest memory footprint. Hence, we can preload their input files to an in-memory filesystem to eliminate the potential influence of disk I/O. We modify these programs so that a Java loader program can invoke the corresponding benchmarking methods via JNI. We first compile the programs into native libraries using the RISC-V clang toolchain. Their benchmarking results can serve as a baseline. After that, we recompile the programs into WebAssembly binaries and rerun the measurements in the following five different settings:

- (1) Interpreter mode: All programs are executed using an interpreter.
- (2) AOT mode: All programs are compiled using our AOT compiler. All sandboxing mechanisms are enabled.
- (3) Based on (2), enable the “`mmap`” optimization.
- (4) Based on (3), skip memory bounds-checking.
- (5) Based on (4), bypass system call protection.

Settings 1 and 2 compare the performance of two runtime implementations: interpreter and AOT compiler. Settings 3 to 5 investigate how WebAssembly sandboxing mechanisms may impact program performance. We execute each benchmark program for 50 times and the average execution time is measured. It is worth mentioning that the WebAssembly compiler toolchain (clang-wasm) provides experimental support to SIMD, bulk memory operations, and tail calls. By default, the compiler disables them as their specifications are yet to be ratified.² In our experiments, we manually turn on these features to pursue a fair comparison with the baseline.

We report the relative execution time in Figure 3. It can be seen that WasmAndroid in the interpreter mode achieves the worst performance; it is averagely 14.83× slower than the baseline. The slowdown is not unexpected, because the interpreter steps through the WebAssembly program only one instruction at a time, which is then decoded and dispatched to the corresponding interpreter routines. This paradigm introduces at least five branching statements for each instruction, thus significantly slowing down the execution. Nevertheless, we argue that the interpreter has its own advantages, especially in some situations where speed is not the primary concern. First, the interpreter code is effortlessly portable to different hardware platforms, since it is implemented in a standard C language. In contrast, porting an AOT compiler demands much more effort, since AOT makes use of low-level CPU machine instructions. Second, the interpreter may benefit resource-constrained devices, since it has a significantly smaller executable size

²They are all phase-3 proposals, meaning they have been implemented and waiting to be merged to the standard specifications.

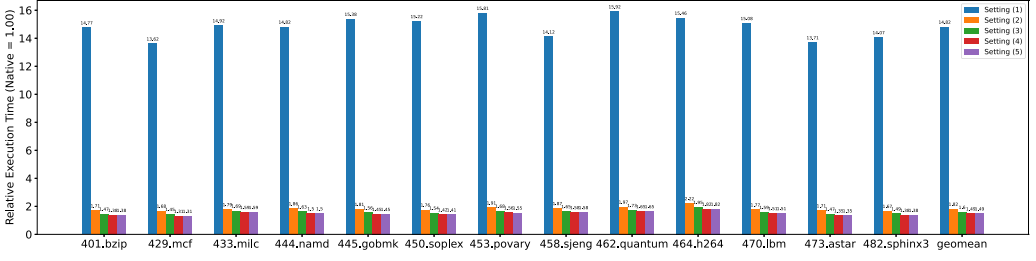


Fig. 3. Performance benchmark of compute-intensive programs.

and memory footprint than an AOT compiler. Furthermore, the interpreter can easily enable single-step debugging on WebAssembly bytecode.

Another important observation is that WasmAndroid in the AOT mode performs $1.82\times$ slower than the baseline averagely. If we enable “mmap” optimization in the setting (3), then we can further witness up to 27% performance boost. This improvement is mainly attributed to two factors as discussed in Section 4.2. First, the “mmap” technique turns every WebAssembly address into a valid OS address, thus bypassing the expensive WebAssembly address transformation process. Second, the “mmap” technique also opens up some opportunities for static bounds-checking elimination to minimize the runtime overhead. Note that this elimination depends on value range analysis, whose results are conservative. In other words, if the analysis is unsure about the range of a particular value, then it will return $[0, \infty]$ so that the bounds check for the value will not be removed. This also justifies the extra performance improvement (up to 14%) when we disable runtime bounds-checking in the setting (4). When we circumvent the system call protection in the setting (5), the performance gain is almost negligible. This is expected, since all the benchmarking programs are compute-intensive and barely issue any system calls. We will investigate how the system call protection impacts real-world kernel-intensive applications in the next section.

To sum up, for CPU-intensive applications, our AOT compiler can provide high application portability while delivering a performance close to native ($1.49\times$ baseline). This performance is arguably acceptable for many applications that are not very performance critical. A number of studies [24] on WebAssembly runtime have provided some insights on the performance gap. One potential culprit is that several WebAssembly arithmetic instructions have special semantics. For example, the operation “ $\text{INT_MIN} \bmod -1$ ” is undefined for most platforms, but WebAssembly specifications define the result as 0. This requires us to insert conditional branching instructions for most modulo operators. Similarly, we also need to insert extra guard conditions on some floating-point instructions to prevent numerical truncation errors. Another possible cause is that our AOT compiler, as a research prototype, still has room for improvement. For example, the precise ordering of optimization passes in the compiler can drastically impact a program’s performance [32]. Currently, we heuristically select an ordering based on our understanding of the characteristics of each pass. The result, however, may still be sub-optimal.

5.3 Real-world Applications

WasmAndroid provides the system call protection mechanism to reduce the attack surface of native code. This experiment investigates its performance efficiency by benchmarking the following real-world kernel-intensive applications:

- (1) **Ping-pong.** Ping-pong carries out a network connectivity test via a pair of UDP sockets between two WebAssembly processes. Each process takes turns to perform 1,500 bytes reads and writes to the other process.

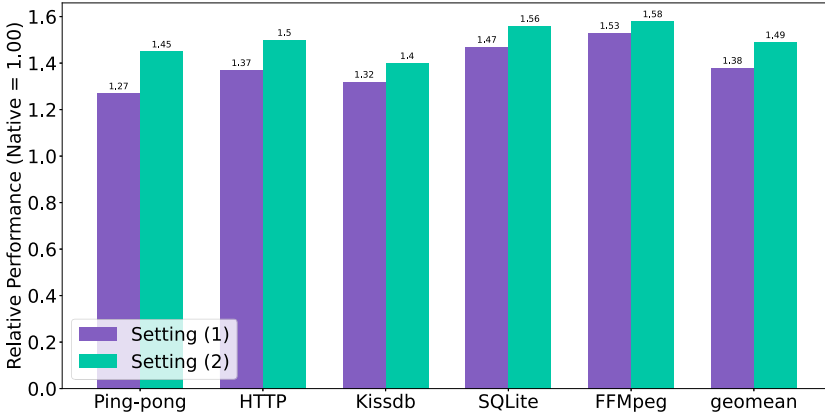


Fig. 4. Performance benchmark of real-world programs.

- (2) **HTTP Server.** Libmicrohttpd [17] is a performant and embeddable HTTP server. The server is configured to run in a single-thread mode. A client runs a stress testing tool called ApacheBench to execute 5,000 HTTP GET requests in total and 500 requests at a time. Each request will obtain a dummy 1-KB file. After each request, the server will close the TCP connection.
- (3) **Key-value Database.** Kissdb [7] is a key-value storage written in C. It stores keys and values of fixed length in a file format based on fixed-size hash tables. The server runs in single-threaded mode. A client runs a stress test by issuing PUT commands to insert one million keys. For each key, the value size is two-byte. After that, the client issues GET commands to retrieve all key-value pairs in random order.
- (4) **SQL Database.** SQLite [33] is a relational database management system contained in a C library. It is widely deployed as local storage in numerous applications (e.g., Web browsers). We benchmark the library by performing 100,000 insertions to an indexed table using transactions.
- (5) **FFMpeg.** FFMpeg [5] is a collection of libraries and tools to process multimedia content such as audio, video, subtitles and related metadata. In this benchmark, we evaluate its h264 video decoding component (i.e., x264 portable) on a 64-MB video clip.

We conduct the measurement on the same machine as the previous experiment. We compile the selected programs into native libraries using the RISC-V clang toolchain. Their benchmarking results will serve as a baseline. After that, we recompile the programs into WebAssembly binaries and rerun the measurements in the following two settings:

- (1) WebAssembly binaries without system call protection (i.e., expose all system calls to WebAssembly's imported function table).
- (2) WebAssembly binaries with system call protection (i.e., expose the Lua-based system call wrappers).

We present the results in Figure 4. Each bar represents the relative performance compared to native binaries. One finding is that native binaries outperform WebAssembly by at least 27%. The performance gap has been justified in the previous section. A more important discovery is that our Lua-based system call protection mechanism incurs averagely 10.60% performance penalty to the applications. A major cause is that the system call wrappers are written in the interpreted language Lua, which is several magnitudes slower than other static languages (e.g., C/C++). As

the system call frequency increases, the performance gaps become more obvious and a higher amount of CPU cycles are spent on interpretation. Nevertheless, we believe the performance cost is acceptable; at this level of performance, most developers targeting our system may do better to tune the source code rather than worry about the overhead. In our continuing work we are pursuing opportunities to further reduce the overhead, for instance, by replacing the Lua interpreter via a just-in-time compiler. Another interesting observation is that Ping-pong received significantly more penalties than FFMpeg. We uncovered a potential cause by measuring the system call time ratio of each application (i.e., fraction of time spent in the kernel rather than in user space) in Setting (1). Ping-pong has the highest ratio (93%), since it demands little computation and frequently invokes network calls. In contrast, FFMpeg has the lowest ratio (3%), because it mostly conducts arithmetic computation for video processing and only issues a limited system calls for I/O.

5.4 Memory Consumption

We reuse all the benchmark programs from the previous sections and measure WasmAndroid's memory overhead. We observe that WasmAndroid demands approximately 82 MB of memory space during the application installation process. This overhead is mainly attributed to the WebAssembly compilation process, which relies on a sophisticated LLVM compiler pipeline. Nevertheless, this overhead is one-off and almost negligible in modern devices considering they usually possess gigabytes of memory space. We also notice that the compiled binaries consume memory space similar ($\pm 2\%$) to their native counterparts in runtime. This finding is not surprising, because the WebAssembly binaries possess programming semantics almost identical to the native binaries.

5.5 Compilation Overhead and Platform Differences

The experiments were not without limitations. First, our AOT compiler takes extra CPU time to compile WebAssembly binaries during the application installation. It is a one-off overhead and does not impact runtime performance and user experience. Our preliminary studies show that the time is roughly proportional to the size of WebAssembly binaries. More research on that is certainly warranted. Second, we conducted our experiments mainly on emulated RISC-V machines. Nevertheless, we have also attempted to synthesize different types CPUs (e.g., ARM and RISC-V) on an FPGA development board (Virtex FPGA VC707 [14]) and deploy WasmAndroid on it. We envision this setup will enable us to thoroughly compare different hardware platforms and obtain more accurate performance benchmarks. Last, the main idea of WasmAndroid is that instead of generating native binaries for every CPU architecture, we can compile native source code into a single representation: WebAssembly. Suppose we have N kinds of target CPU architectures, we expect to reduce the compilation effort by N times. We plan to conduct a user study to quantitatively measure compilation workload using the FPGA platform.

6 SYSTEM EXTENSIONS AND LIMITATIONS

WasmAndroid allows native Android applications to be compiled into WebAssembly and run on devices with different hardware architectures. Nevertheless, WasmAndroid is still a research prototype and bears certain limitations. In this section, we suggest several future research directions.

6.1 Static Translate Binary Files

We have been investigating the possibility of transpiling an existing binary file (presumably targeting ARM processors) to WebAssembly. This could be beneficial when developers want to port an application to different platforms but do not have access to source files. In this article, we implemented a static translator, which is highly motivated by several previous research works

[20, 35, 44]. The core component is a binary raiser that translates machine code to LLVM IR, which later can be fed into LLVM backends to generate WebAssembly. The binary raising process consists of several phases. First, the binary is parsed and the text section is disassembled as a sequence of low-level machine instructions. Afterward, this code sequence is partitioned as functions based on symbol tables in the binary. Then we construct the control flow graph for each function. By walking the graph, we can reconstruct all basic blocks and the function prototype. Finally, by analyzing all basic blocks, we can raise each instruction to LLVM IR.

This static translator is still a highly experimental product. Similar to existing static translators, our implementation is yet to overcome several typical challenges such as code discovery (i.e., distinguishing data from instructions) and indirect branching (i.e., the jump destination address must be mapped to an address in the translated code). We plan to devote more effort to translator implementation in the future.

6.2 32-bit vs. 64-bit

WebAssembly comes in two flavors: 32-bit (wasm32) and 64-bit (wasm64). Compared with wasm32, wasm64 provides no new instructions but instead extends the memory instructions to use 64-bit integers as operands. In other words, a pointer in wasm64 is 64-bit and can address 2^{64} bytes of memory. In our prototype, we select the 64-bit flavor to facilitate data exchange between Java runtime and WebAssembly. In detail, native methods sometimes can receive opaque data structures (e.g., JObject and JClass) from the Android Java runtime. These data structures serve as handles to Java resources and are essential aliases of ordinary pointers (i.e., typedef void* JObject). Since the modern Android Java runtime runs in the 64-bit mode, these opaque pointers are 64-bit wide. Hence, it is natural to use wasm64, which provides intrinsic support to 64-bit pointers.

Nevertheless, pointers take twice as much space on 64-bit platforms than on 32-bit platforms. It means wasm64 will almost certainly consume more memory than wasm32. Recently, we have been investigating the potential of wasm32 for resource-constrained devices. To correctly handle 64-bit pointers in wasm32, we patch several C/C++ header files. A major modification is to rewrite the definitions of JObject from “typedef void* JObject” to “typedef long long JObject.” In addition, we enhance the function transformation wrapper as discussed in Listing 5; the size of “dupenv” should reduce to half, since elements inside shrink from 64-bit to 32-bit.

6.3 Alternative Intermediate Representations

In this article, we select WebAssembly as the intermediate representation for native source code, because it is fully agnostic to target platforms. It also offers source code protection and execution performance close to native binaries. Nevertheless, we have also been investigating alternative intermediate representations. One candidate is LLVM IR [29], a low-level programming language similar to assembly. LLVM IR is a strongly typed RISC. It is capable of representing the semantics of many static programming languages such as C and C++. Despite its popularity, LLVM IR has a significant drawback; It is not fully independent of the target machines if the source language specification is interpreted differently on different platforms. One notorious example is that the long type in C may be mapped to the LLVM type i32 or i64, depending on whether the source code is compiled for a 32-bit or a 64-bit machine. This again leads to the portability issue. Another portable intermediate representation we have attempted is Dalvik bytecode [4]. Dalvik contains a variety of **complex instruction set computing instructions (CISC)**. For example, the instruction “check-cast vx, typeid” will check whether the object reference in vx can be cast to an instance of a class referenced by typeid. If the cast is not possible, then this instruction throws ClassCastException. Executing such CISC instructions requires sophisticated runtime functionalities

(e.g., automatic memory management and type introspection). As a result, the runtime implementation may demand more engineering efforts.

6.4 WebAssembly Toolchain Limitations

As we discussed in Section 4.4, native android applications make heavy use of POSIX functions in standard libraries (e.g., `libc` and `libc++`). WebAssembly toolchains also provide an `libc` implementation named WASI-`libc`. Although WASI-`libc` aims to conform to the POSIX standards, a considerable number of functions are still missing. For example, WASI-`libc` only provides partial support to networking and asynchronous IO. In addition, WASI-`libc` does not support `setjmp/longjmp` and C++ exceptions, since they require a stack unwinding scheme. C++ code, for now, can only be compiled only with the compiler flag “`-fno-exceptions`” (i.e., applications will trap when an exception happens). WASI-`libc` is also yet to support process management system calls (e.g., `fork`). Nevertheless, WebAssembly is a fast-evolving technology and considerable research effort is being devoted to removing the limitations we mentioned above. For instance, the proposal to support stack unwinding and exception handling is being ratified and will be available soon [10]. A stack switching proposal is also submitted, which lays the foundation of process management support in future WebAssembly versions [12]. In our implementation, we also enhance the wasi-`libc` implementation to provide a set of commonly-used networking functions such as *connect*, *accept*, and *listen*.

6.5 Security Implications

Address space layout randomization (ASLR) is a system security mechanism that guards against memory corruption vulnerabilities by randomizing processes’ address space such as stacks, heaps, and libraries. WasmAndroid provides preliminary ASLR support by randomizing the address of WebAssembly linear memory. This, however, inhabits several compiler optimizations such as `mmap` and bounds-check removal. We plan to devote more effort to improving the compiler optimizer so that WasmAndroid can enforce ASLR without sacrificing significant runtime performance.

WebAssembly is low-level machine instructions, which are significantly harder to reverse-engineer than high-level Java bytecode. Nevertheless, developers can apply an extra layer of protection to their WebAssembly binary files via code obfuscators in WebAssembly toolchains. An exemplary tool is Obfuscator-LLVM [25], which provides efficient obfuscating code transformations such as instruction substitutions, bogus control-flow insertion, basic-block splitting, and control-flow flattening.

7 CONCLUSION

This article presents WasmAndroid, a performant and portable runtime for native Android applications. WasmAndroid enables developers to compile their native source code into WebAssembly once and execute it on every hardware platform without additional reconfiguration. WebAssembly features a language model very different from other common programming languages. This mismatch leads to many unique implementation challenges. In this article, we present workable solutions and conduct a thorough system evaluation. Our results show that WasmAndroid achieves comparable performance to their native counterparts.

REFERENCES

- [1] Android Has Been Ported to a RISC-V Board. Retrieved from <https://www.xda-developers.com/android-risc-v-port/>.
- [2] Android Runtime (ART) and Dalvik. Retrieved from <https://source.android.com/devices/tech/dalvik>.
- [3] Clang: A C Language Family Frontend for LLVM. Retrieved from <https://clang.llvm.org/>.

- [4] Dalvik bytecode. Retrieved from <https://source.android.com/devices/tech/dalvik/dalvik-bytecode>.
- [5] FFmpeg: A Cross-platform Solution to Record, Convert and Stream Audio and Video. Retrieved from <https://github.com/FFmpeg/FFmpeg>.
- [6] How Alibaba Is Porting RISC-V to the Android OS. Retrieved from <https://riscv.org/blog/2021/11/how-alibaba-is-porting-risc-v-to-the-android-os-guoyin-chen-alibaba/>.
- [7] Kissdb: Keep It Simple Stupid Database. Retrieved from <https://github.com/adamierymenko/kissdb>.
- [8] POWER Instruction Set Architecture (ISA). Retrieved from <https://openpowerfoundation.org/tag/microwatt/>.
- [9] THead Android RISC-V Port. Retrieved from <https://github.com/T-head-Semi/aosp-riscv>.
- [10] WebAssembly Exception Handling Proposal. Retrieved from <https://github.com/WebAssembly/exception-handling>.
- [11] WebAssembly SIMD proposal. Retrieved from <https://v8.dev/features/simd>.
- [12] WebAssembly Stack Switching Proposal. Retrieved from <https://github.com/WebAssembly/stack-switching>.
- [13] The WebAssembly System Interface. Retrieved from <https://wasi.dev/>.
- [14] Xilinx Virtex-7 FPGA VC707 Evaluation Kit. Retrieved from <https://www.xilinx.com/products/boards-and-kits/ek-v7-vc707-g.html>.
- [15] Fabrice Bellard. 2005. QEMU, a fast and portable dynamic translator. In *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*, Vol. 41. 46.
- [16] Pau Oliva Fora. 2014. Beginners guide to reverse engineering android apps. In *Proceedings of the RSA Conference*. 21–22.
- [17] C. Grothoff. 2014. GNU Libmicrohttpd: A Library for Creating an Embedded http Server. Retrieved from <https://www.gnu.org/software/libmicrohttpd/>.
- [18] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and J. F. Bastien. 2017. Bringing the web up to speed with WebAssembly. In *ACM SIGPLAN Notices*, Vol. 52. ACM, 185–200.
- [19] Adam Hall and Umakishore Ramachandran. 2019. An execution model for serverless functions at the edge. In *Proceedings of the International Conference on Internet of Things Design and Implementation*. ACM, 225–236.
- [20] Joe Hendrix, Guannan Wei, and Simon Winwood. 2019. Towards verified binary raising. In *Workshop on Instruction Set Architecture Specification (co-located with ITP 2019)*. 6.
- [21] John L. Henning. 2006. SPEC CPU2006 benchmark descriptions. *ACM SIGARCH Comput. Arch. News* 34, 4 (2006), 1–17.
- [22] Roberto Ierusalimsky. 2006. *Programming in Lua*. Roberto Ierusalimsky.
- [23] Martin Jacobsson and Jonas Wåhslén. 2018. Virtual machine execution for wearables based on WebAssembly. In *Proceedings of the 13th EAI International Conference on Body Area Networks (BodyNets'18)*.
- [24] Abhinav Jangda, Bobby Powers, Emery D. Berger, and Arjun Guha. 2019. Not so fast: Analyzing the Performance of WebAssembly vs. native code. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC'19)*. 107–120.
- [25] Pascal Junod, Julien Rinaldini, Johan Wehrli, and Julie Michielin. 2015. Obfuscator-LLVM—software protection for the masses. In *Proceedings of the IEEE/ACM 1st International Workshop on Software Protection*. IEEE, 3–9.
- [26] Uday P. Khedker, Amitabha Sanyal, and Bageshri Karkare. 2017. *Data Flow Analysis: Theory and Practice*. CRC Press.
- [27] Yeong-Jun Kim, Seong-Jin Cho, Kil-Jae Kim, Eun-Hye Hwang, Seung-Hyun Yoon, and Jae-Wook Jeon. 2012. Benchmarking Java application using JNI and native C application on Android. In *Proceedings of the 12th International Conference on Control, Automation and Systems*. IEEE, 284–288.
- [28] Radhesh Krishnan Konoth, Emanuele Vineti, Veelasha Moonsamy, Martina Lindorfer, Christopher Kruegel, Herbert Bos, and Giovanni Vigna. 2018. Minesweeper: An in-depth look into drive-by cryptocurrency mining and its defense. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. ACM, 1714–1730.
- [29] Chris Lattner. 2008. LLVM and Clang: Next generation compiler technology. In *Proceedings of the BSD Conference*, Vol. 5.
- [30] John Levine. 2009. *Flex & Bison: Text Processing Tools*. O'Reilly Media, Inc.
- [31] Bruno Gois Mateus and Matias Martinez. 2019. An empirical study on quality of Android applications written in Kotlin language. *Emp. Softw. Eng.* 24, 6 (2019), 3356–3393.
- [32] Ricardo Nobre, Luiz G. A. Martins, and João M. P. Cardoso. 2016. A graph-based iterative compiler pass selection and phase ordering approach. *ACM SIGPLAN Not.* 51, 5 (2016), 21–30.
- [33] Mike Owens. 2006. *The Definitive Guide to SQLite*. Apress.
- [34] Sylvain Ratabouil. 2015. *Android NDK: Beginner's Guide*. Packt Publishing Ltd.
- [35] Bor-Yeh Shen, Wei-Chung Hsu, and Wu Yang. 2014. A retargetable static binary translator for the ARM architecture. *ACM Trans. Arch. Code Optim.* 11, 2 (2014), 1–25.
- [36] Sajjad Taheri, Alexander Vedenbaum, Alexandru Nicolau, Ningxin Hu, and Mohammad R. Haghighat. 2018. OpenCV.js: Computer vision processing for the open Web platform. In *Proceedings of the 9th ACM Multimedia Systems Conference*. ACM, 478–483.
- [37] James Tandon. 2011. The openrisc processor: Open hardware and linux. *Linux J.* 2011, 212 (2011), 6.

- [38] Andrew Shell Waterman. 2016. *Design of the RISC-V Instruction Set Architecture*. Ph.D. Dissertation. UC Berkeley.
- [39] Fengguo Wei, Xingwei Lin, Xinming Ou, Ting Chen, and Xiaosong Zhang. 2018. Jn-saf: Precise and efficient ndk/jni-aware inter-language static analysis framework for security vetting of android applications with native code. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. 1137–1150.
- [40] Elliott Wen, Jim Warren, and Gerald Weber. 2020. BrowserVM: Running unmodified operating systems and applications in browsers. In *Proceedings of the IEEE International Conference on Web Services (ICWS'20)*. IEEE, 473–480.
- [41] Elliott Wen and Gerald Weber. 2020. Wasmachine: Bring IoT up to speed with A WebAssembly OS. In *Proceedings of the IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*. IEEE, 1–4.
- [42] Elliott Wen, Gerald Weber, and Suranga Nanayakkara. 2021. WasmAndroid: A cross-platform runtime for native programming languages on Android (WIP paper). In *Proceedings of the 22nd ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*. 80–84.
- [43] R. Winsniewski. 2012. Android–apktool: A tool for reverse engineering android apk files. <https://ibotpeaches.github.io/Apktool/>.
- [44] S. Bharadwaj Yadavalli and Aaron Smith. 2019. Raising binaries to LLVM IR with MCTOLL (WIP paper). In *Proceedings of the 20th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*. 213–218.
- [45] Alon Zakai. 2018. Fast physics on the web using C++, JavaScript, and emscripten. *Comput. Sci. Eng.* 20, 1 (2018), 11–19.

Received 8 October 2021; revised 13 February 2022; accepted 3 April 2022